

Report 1^ο Εργαστηρίου - Λειτουργικά Συστήματα

Υπολογιστών

Μέρος 1

- Ζητούμενο:

Υλοποίηση ενός προγράμματος το οποίο θα αναζητά πόσες φορές εμφανίζεται ένας χαρακτήρας μέσα σε ένα αρχείο εισόδου και θα γράφει το αποτέλεσμα σε ένα αρχείο εξόδου, μόνο με τη χρήση system-calls.

Τόσο τα αρχεία, όσο και ο χαρακτήρας προς αναζήτηση, θα δίνονται από τον χρήστη στη γραμμή εντολών.

Έτσι ώστε να μην δημιουργείται πρόβλημα αν ο χρήστης δεν δώσει τις παραμέτρους στη γραμμή εντολών σωστά, το πρόγραμμά θα πραγματοποιεί τους απαραίτητους ελέγχους και θα δίνει τις κατάλληλες οδηγίες χρήσης.

- Κώδικας

```
1  ▶ #include <fcntl.h>
2  ▶ #include <unistd.h>
3  ▶ #include <stdlib.h>
4  ▶ #include <stdio.h>
5
6  ▶ int main(int argc, char *argv[]) {
7      if (argc != 4) {
8          write(STD_2, Buf: "Usage: program input output char\n", nbytes: 33);
9          exit(1);
10     }
11     int fd_in = open(argv[1], O_RDONLY);
12     if (fd_in < 0) {
13         perror("open input");
14         exit(1);
15     }
16     int fd_out = open(argv[2], O_WRONLY | O_CREAT | O_TRUNC, 0644);
17     if (fd_out < 0) {
18         perror("open output");
19         exit(1);
20     }
21     char buffer[1024];
22     ssize_t bytes;
23     char target = argv[3][0];
24     int count = 0;
25     while ((bytes = read(fd_in, buffer, nbytes: sizeof(buffer))) > 0) {
26         for (int i = 0; i < bytes; i++) {
27             if (buffer[i] == target) {
28                 count++;
29             }
30         }
31     }
32     if (bytes < 0) {
33         perror("read");
34         exit(1);
35     }
36     char result[50];
37     int len = sprintf(result, "Count: %d\n", count);
38     write(fd_out, result, len);
39     close(fd_in);
40     close(fd_out);
41     return 0;
42 }
```

- Επεξήγηση κώδικα

Το πρόγραμμα διαβάζει ένα αρχείο εισόδου, μετράει πόσες φορές εμφανίζεται ένας συγκεκριμένος χαρακτήρας και γράφει το αποτέλεσμα σε ένα αρχείο εξόδου, χρησιμοποιώντας αποκλειστικά system calls. Δηλαδή, χαμηλού επιπέδου κλήσεις συστήματος αντί για τις κλασσικές εντολές τις C, "fopen" και "fclose"

- Πετυχαίνει τον στόχο του;

ΝΑΙ!

- Επεξήγηση τμημάτων του κώδικα

Έλεγχος Ορισμάτων

```
if (argc != 4) {  
    write(2, "Usage: program input output char\n", 33);  
    exit(1);  
}
```

Το πρόγραμμα απαιτεί ακριβώς 3 ορίσματα:

- argv[1] → αρχείο εισόδου
- argv[2] → αρχείο εξόδου
- argv[3] → ο χαρακτήρας προς αναζήτηση

Το argc μετράει και το ίδιο το όνομα του προγράμματος (argv[0]), οπότε ελέγχεται για 4. Αν δεν ισχύει, γράφει μήνυμα σφάλματος στο stderr και τερματίζει.

Άνοιγμα Αρχείων

```
int fd_in = open(argv[1], O_RDONLY);
```

Ανοίγει το αρχείο εισόδου σε κατάσταση μόνο ανάγνωση. Επιστρέφει έναν ακέραιο, τον file descriptor (fd_in). Αν αποτύχει, επιστρέφει -1.

```
int fd_out = open(argv[2], O_WRONLY | O_CREAT | O_TRUNC, 0644);
```

Ανοίγει/δημιουργεί το αρχείο εξόδου με τρεις flags

O_WRONLY: Μόνο εγγραφή

O_CREAT: Δημιουργία αν δεν υπάρχει

O_TRUNC: Αδειάζει το αρχείο αν ήδη υπάρχει

0644: Τα δικαιώματα 0644 σημαίνουν: ο ιδιοκτήτης έχει read+write, οι υπόλοιποι μόνο read.

Κύριος Βρόχος Ανάγνωσης

```
char buffer[1024];
```

```
ssize_t bytes;
```

```
char target = argv[3][0];
```

```
int count = 0;
```

```
while ((bytes = read(fd_in, buffer, sizeof(buffer))) > 0) {
```

```
    for (int i = 0; i < bytes; i++) {
```

```
        if (buffer[i] == target) {
```

```
            count++;
```

```
        }
```

```
    }
```

```
}
```

- ο target είναι ο πρώτος χαρακτήρας του argv[3]
- Η read() διαβάζει έως 1024 bytes κάθε φορά και επιστρέφει πόσα bytes διάβασε πραγματικά
- Ο εσωτερικός βρόχος εξετάζει κάθε byte του buffer και αυξάνει τον count αν ταιριάζει
- Ο βρόχος σταματά όταν η read() επιστρέψει 0 (τέλος αρχείου) ή αρνητικό (σφάλμα)

Με αυτή τη λογική διαχειρίζεται σωστά και αρχεία μεγαλύτερα των 1024 bytes, αφού διαβάζει σε chunks.

Εγγραφή Αποτελέσματος

```
char result[50];
```

```
int len = sprintf(result, "Count: %d\n", count);
```

```
write(fd_out, result, len);
```

Μορφοποιεί το αποτέλεσμα σε string (π.χ. "Count: 42\n") και το γράφει στο αρχείο εξόδου.

Παράδειγμα Εκτέλεσης

```
./program input.txt output.txt 'a'
```

Μετράει πόσες φορές εμφανίζεται το 'a' στο input.txt και γράφει το αποτέλεσμα στο output.txt.

- System Calls που χρησιμοποιεί ο κώδικας
 - open()
 - read()
 - write()
 - close()
 - exit()

Μέρος 2.1

- Ζητούμενο:

Να επεκταθεί το πρόγραμμα του 1^{ου} μέρους, ώστε:

1. Να δημιουργηθεί μία διεργασία παιδί που θα "χαιρετάει τον κόσμο" αναφέροντας το αναγνωριστικό της και το αναγνωριστικό του γονέα της.
2. Ο γονέας να τυπώνει το αναγνωριστικό του παιδιού και να περιμένει τον τερματισμό του.
3. Να οριστεί μια μεταβλητή x, στο γονέα πριν τη δημιουργία του παιδιού, και να ανατεθεί διαφορετική τιμή στο γονέα και στο παιδί και να τυπωθεί η τιμή της.
4. Να ανατεθεί στη διεργασία παιδί, η αναζήτηση του χαρακτήρα στο αρχείο.
5. Να δημιουργηθεί μία διεργασία παιδί που να εκτελεί τον κώδικα που δόθηκε στο Ερώτημα 1.

- Κώδικας 2.1

```
1 #include <fcntl.h>
2 #include <unistd.h>
3 #include <stdlib.h>
4 #include <stdio.h>
5 #include <sys/wait.h>
6
7 int main(int argc, char *argv[]) {
8     if (argc != 4) {
9         write(STDOUT_FILENO, "Usage: program input output char\n", 33);
10        exit(1);
11    }
12
13    int fd_in = open(argv[1], O_RDONLY);
14    if (fd_in < 0) {
15        perror("open input");
16        exit(1);
17    }
18
19    int fd_out = open(argv[2], O_WRONLY | O_CREAT | O_TRUNC, 0644);
20    if (fd_out < 0) {
21        perror("open output");
22        exit(1);
23    }
24
25    int x = 10;
26    printf("[Parent] Variable x initialized to: %d\n", x);
27    pid_t pid = fork();
28    if (pid < 0) {
29        perror("fork failed");
30        exit(1);
31    }
32    else if (pid == 0) {
33        printf("[Child] Hello World! My PID is %d. My parent's PID is %d\n", getpid(), getppid());
34        x = 20;
35        printf("[Child] I changed x to: %d\n", x);
36        char buffer[1024];
37        ssize_t bytes;
38        char target = argv[3][0];
39        int count = 0;
40
41        while ((bytes = read(fd_in, buffer, sizeof(buffer))) > 0) {
42            for (int i = 0; i < bytes; i++) {
43                if (buffer[i] == target) {
44                    count++;
45                }
46            }
47
48            if (bytes < 0) {
49                perror("read");
50                exit(1);
51            }
52
53            char result[50];
54            int len = sprintf(result, "Count: %d\n", count);
55            write(fd_out, result, len);
56            exit(0);
57        }
58    }
59    else {
60        printf("[Parent] I created the child whose PID is: %d\n", pid);
61        x = 30;
62        printf("[Parent] I changed x to: %d\n", x);
63        wait(NULL);
64        printf("[Parent] Child, terminated.\n");
65        close(fd_in);
66        close(fd_out);
67    }
68    return 0;
69 }
```

- Επεξήγηση κώδικα

Ο κώδικας είναι εξελιγμένη έκδοση του προηγούμενου. Η βασική λογική του, δηλαδή η μέτρηση χαρακτήρα παραμένει, αλλά τώρα εκτελείται από ένα child process που δημιουργείται με fork().

- Πετυχαίνει τον στόχο του;

ΝΑΙ!

- Επεξήγηση τμημάτων του κώδικα

Άνοιγμα Αρχείων

```
int fd_in = open(argv[1], O_RDONLY);
```

```
int fd_out = open(argv[2], O_WRONLY | O_CREAT | O_TRUNC, 0644);
```

Τα αρχεία ανοίγουν από τον parent πριν το fork(). Αυτό σημαίνει ότι και τα δύο processes μοιράζονται τους ίδιους file descriptors μετά το fork.

Η Μεταβλητή x και η Έννοια του Virtual Memory

```
int x = 10;
```

```
printf("[Parent] Variable x initialized to: %d\n", x);
```

```
pid_t pid = fork();
```

Η μεταβλητή x αρχικοποιείται πριν το fork(), οπότε και τα δύο processes ξεκινούν με x = 10. Στη συνέχεια, παρόλο που φαίνεται ότι μοιράζονται την ίδια μεταβλητή, δεν συμβαίνει αυτό. Το λειτουργικό χρησιμοποιεί Copy-on-Write (CoW) και η μνήμη αντιγράφεται μόνο όταν κάποιο process τη τροποποιήσει. Έτσι κάθε process έχει το δικό του αντίγραφο του x.

Το fork() και οι Τρεις Περιπτώσεις

```
pid_t pid = fork();
```

```
if (pid < 0) { /* Αποτυχία */ }
```

```
else if (pid == 0) { /* Child process */ }
```

```
else { /* Parent process */ }
```

Γιατί το wait(NULL) Είναι Απαραίτητο

Αν ο parent δεν περίμενε το child, τότε το child θα γινόταν Zombie process. Δηλαδή τερματισμένο αλλά η καταχώρησή του στον process table παραμένει μέχρι ο parent να διαβάσει τον κωδικό εξόδου. Χρησιμοποιώντας το wait(NULL), ο parent αγνοεί τον κωδικό εξόδου αλλά αποτρέπει το zombie.

- System Calls που χρησιμοποιεί ο κώδικας

- open()
- read()
- write()
- close()
- exit()
- fork()
- getpid()
- getppid()
- wait(NULL)

- Κώδικας 2.2

```
1  #include <unistd.h>
2  #include <stdlib.h>
3  #include <stdio.h>
4  #include <sys/wait.h>
5
6  int main(int argc, char *argv[]) {
7      if (argc != 4) {
8          printf("Usage: %s input output char\n", argv[0]);
9          exit(1);
10     }
11     pid_t pid = fork();
12     if (pid < 0) {
13         perror("fork failed");
14         exit(1);
15     }
16     else if (pid == 0) {
17         char *args[] = {"/part1", argv[1], argv[2], argv[3], NULL};
18         execv(path: "/part1", args);
19         perror("execv failed");
20         exit(1);
21     }
22     else {
23         wait(NULL);
24         printf("[Parent] The child (through syscall: execv) has completed the execution.\n");
25     }
26     return 0;
27 }
```

- Επεξήγηση κώδικα

Ο κώδικας αυτός είναι ένα launcher. Ο parent δημιουργεί ένα child process το οποίο αντικαθιστά τον εαυτό του με ένα άλλο πρόγραμμα (./part1) χρησιμοποιώντας το system call `execv()`.

- Πετυχαίνει τον στόχο του;

ΝΑΙ!

- Επεξήγηση τμημάτων του κώδικα

Έλεγχος Ορισμάτων

```
if (argc != 4) {  
    printf("Usage: %s input output char\n", argv[0]);  
    exit(1);  
}
```

Ίδιος έλεγχος με πριν. Απαιτούνται 3 ορίσματα. Εδώ χρησιμοποιείται `printf` αντί για `write()` στο `stderr`.

fork() - Δημιουργία Child

```
pid_t pid = fork();
```

Δημιουργείται ένα αντίγραφο της τρέχουσας διεργασίας. Από αυτό το σημείο εκτελούνται δύο ανεξάρτητες διεργασίες.

Child Process - execv()

```
char *args[] = {"/part1", argv[1], argv[2], argv[3], NULL};  
execv("./part1", args);  
perror("execv failed");  
exit(1);
```

Το `execv()` αντικαθιστά πλήρως το τρέχον process image με ένα νέο πρόγραμμα. Αυτό σημαίνει πως ο κώδικας, η στοίβα και το heap, αντικαθίστανται. Το PID παραμένει το ίδιο και δεν δημιουργείται νέα διεργασία. Τέλος, δεν επιστρέφει ποτέ αν πετύχει. Αν φτάσει στη `perror()`, σημαίνει ότι απέτυχε.

Array() - args[]

```
char *args[] = {"/part1", argv[1], argv[2], argv[3], NULL};  
//      ↑ argv[0] ↑ input ↑ output ↑ char ↑ τερματισμός
```

Το NULL στο τέλος είναι απαραίτητο έτσι ώστε το `execv()` να ξέρει πού τελειώνουν τα ορίσματα.

- System Calls που χρησιμοποιεί ο κώδικας
 - `execv()`
 - `wait(NULL)`

Μέρος 3

- Ζητούμενο:

Να επεκταθεί το πρόγραμμα του Μέρους 2, ώστε να δημιουργεί P διεργασίες παιδιά οι οποίες θα αναζητούν παράλληλα το χαρακτήρα στο αρχείο, ενώ η γονεϊκή διεργασία θα συλλέγει και θα τυπώνει το συνολικό αποτέλεσμα. Όταν το πρόγραμμά σας θα δέχεται Control+C από το πληκτρολόγιο (SIGINT signal) θα πρέπει αντί να τερματίζει, να τυπώνει το συνολικό αριθμό διεργασιών που αναζητούν το αρχείο.

- Κώδικας

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <unistd.h>
4  #include <sys/wait.h>
5  #include <fcntl.h>
6  #include <signal.h>
7
8  #define P 5
9
10 volatile sig_atomic_t active_children = 0;
11
12 void safe_print_int(int num) {
13     char buf[32];
14     int i = 0;
15     if (num == 0) {
16         write(STDOUT_FILENO, buf, "0", sizeof 1);
17         return;
18     }
19     while (num > 0) {
20         buf[i++] = (num % 10) + '0';
21         num /= 10;
22     }
23     for (int j = i - 1; j >= 0; j--) {
24         write(STDOUT_FILENO, &buf[j], sizeof 1);
25     }
26 }
27
28 void sigint_handler(int sig) {
29     write(STDOUT_FILENO, "\n[SIGNAL] Control+C caught! Active search processes: ", sizeof 53);
30     safe_print_int(active_children);
31     write(STDOUT_FILENO, "\n", sizeof 1);
32 }
33
34 int main(int argc, char *argv[]) {
35     if (argc != 4) {
36         write(STDERR_FILENO, "Usage: ./askis3 input_file output_file char\n", sizeof 45);
37         exit(1);
38     }
39
40     struct sigaction sa;
41     sa.sa_handler = sigint_handler;
42     sigemptyset(&sa.sa_mask);
43     sa.sa_flags = SA_RESTART;
44     sigaction(SIGINT, &sa, NULL);
45
46     int pipefd[2];
47     if (pipe(pipefd) == -1) {
48         perror("pipe failed");
49         exit(1);
50     }
51
52     int fd_temp = open(argv[1], O_RDONLY);
53     if (fd_temp < 0) {
54         perror("open input");
55         exit(1);
56     }
57     off_t file_size = lseek(fd_temp, 0, SEEK_END);
58     close(fd_temp);
59
60     off_t chunk_size = file_size / P;
61
62     printf("[PARENT] Starting creation of %d children...\n", P);
63     printf("[PARENT] File size is %ld bytes. You have 5 seconds to press Ctrl+C!\n", (long)file_size);
64
65     for (int i = 0; i < P; i++) {
66         pid_t pid = fork();
67
68         if (pid < 0) {
69             perror("fork failed");
70             exit(1);
71         }
72         else if (pid == 0) {
73             signal(SIGINT, SIG_IGN);
74             close(pipefd[0]);
75
76             sleep(5);
77
78             int fd_in = open(argv[1], O_RDONLY);
79
80             off_t start = i * chunk_size;
81             off_t end = (i == P - 1) ? file_size : start + chunk_size;
82
83             lseek(fd_in, start, SEEK_SET);
84
85             char buffer[1024];
86             ssize_t bytes;
87             char *target = argv[3][0];
88             int local_count = 0;
89             off_t bytes_to_read = end - start;
90             off_t total_read = 0;
91
92             while (total_read < bytes_to_read) {
93                 size_t to_read = sizeof(buffer);
94                 if (bytes_to_read - total_read < sizeof(buffer)) {
95                     to_read = bytes_to_read - total_read;
96                 }
97
98                 bytes = read(fd_in, buffer, to_read);
99                 if (bytes <= 0) break;

```

```

101         for (int j = 0; j < bytes; j++) {
102             if (buffer[j] == target) local_count++;
103         }
104         total_read += bytes;
105     }
106     close(fd_in);
107
108     write(pipefd[1], &local_count, sizeof(int));
109     close(pipefd[1]);
110     exit(0);
111 }
112 else {
113     active_children++;
114 }
115 }
116 close(pipefd[1]);
117
118 int total_count = 0;
119 int partial_count;
120
121 for (int i = 0; i < P; i++) {
122     if (read(pipefd[0], &partial_count, sizeof(int)) > 0) {
123         total_count += partial_count;
124     }
125 }
126 close(pipefd[0]);
127
128 pid_t pid;
129 int status;
130 while ((wpid = wait(&status)) > 0) {
131     active_children--;
132 }
133
134 int fd_out = open(argv[2], O_WRONLY | O_CREAT | O_TRUNC, 0644);
135 if (fd_out >= 0) {
136     char result_msg[50];
137     int len = sprintf(result_msg, "Total Count: %d\n", total_count);
138     write(fd_out, result_msg, len);
139     close(fd_out);
140 }
141
142 printf("[PARENT] Search completed. Found %d occurrences.\n", total_count);
143
144 return 0;
145 }

```

- Επεξήγηση κώδικα

Ο κώδικας υλοποιεί παράλληλη μέτρηση χαρακτήρα σε αρχείο, χωρίζοντάς το σε P (P = 3) τμήματα που επεξεργάζονται ταυτόχρονα από 3 child processes. Επιπλέον, χειρίζεται το σήμα SIGINT (Ctrl+C).

- Πετυχαίνει τον στόχο του;

ΝΑΙ!

- Επεξήγηση τμημάτων του κώδικα

Καθολικές Μεταβλητές & volatile sig_atomic_t

```
#define P 3
```

```
volatile sig_atomic_t active_children = 0;
```

P: Ο αριθμός των child processes, άρα των τμημάτων του αρχείου
volatile sig_atomic_t: ο ειδικός τύπος για τις μεταβλητές που τροποποιούνται μέσα σε signal handlers. Το volatile εμποδίζει τον compiler να την κάνει cache σε register, και το sig_atomic_t εγγυάται atomic (αδιαίρετη) ανάγνωση/εγγραφή.

safe_print_int() — Async-Signal-Safe Εκτύπωση

```
void safe_print_int(int num) {  
    char buf[32];  
    int i = 0;  
    while (num > 0) {  
        buf[i++] = (num % 10) + '0';  
        num /= 10;  
    }  
    for (int j = i - 1; j >= 0; j--)  
        write(STDOUT_FILENO, &buf[j], 1);  
}
```

Εκτυπώνει έναν ακέραιο χρησιμοποιώντας μόνο write() και όχι printf(). Η printf() δεν είναι async-signal-safe, και έτσι, αν διακοπεί από signal ενώ χρησιμοποιεί εσωτερικά locks για το buffer της, μπορεί να προκαλέσει deadlock. Μέσα σε signal handler επιτρέπονται μόνο async-signal-safe συναρτήσεις, όπως η write().

Signal Handler για SIGINT

```
void sigint_handler(int sig) {  
    write(STDOUT_FILENO, "\n[SIGNAL] Control+C caught! Active search processes: ", 53);  
    safe_print_int(active_children);  
    write(STDOUT_FILENO, "\n", 1);  
}
```

Όταν ο χρήστης πατήσει Ctrl+C, Ο parent δεν τερματίζει, απλώς εκτυπώνει πόσα children εκτελούνται ακόμα. Τα children αγνοούν το SIGINT.

Εγκατάσταση Signal Handler με sigaction()

```
struct sigaction sa;  
sa.sa_handler = sigint_handler;  
sigemptyset(&sa.sa_mask);  
sa.sa_flags = SA_RESTART;
```

```
sigaction(SIGINT, &sa, NULL);
```

Πεδίο	Τιμή	Σημασία
sa_handler	sigint_handler	Η συνάρτηση που θα κληθεί
sa_mask	κενή	Κανένα επιπλέον signal δεν μπλοκάρεται κατά τη διάρκεια του handler
sa_flags = SA_RESTART		Οι system calls που διακόπηκαν από το signal επανεκκινούν αυτόματα αντί να επιστρέψουν σφάλμα

Η sigaction() είναι προτιμότερη από την signal() γιατί έχει πιο προβλέψιμη συμπεριφορά across platforms.

Δημιουργία Pipe

```
int pipefd[2];
```

```
pipe(pipefd);
```

```
// pipefd[0] = read end
```

```
// pipefd[1] = write end
```

Το pipe είναι ο μηχανισμός επικοινωνίας (IPC) μεταξύ children και parent. Κάθε child γράφει τον τοπικό του μετρητή στο pipe, και ο parent τους διαβάζει και τους αθροίζει.

Υπολογισμός Μεγέθους Αρχείου με lseek()

```
int fd_temp = open(argv[1], O_RDONLY);
```

```
off_t file_size = lseek(fd_temp, 0, SEEK_END);
```

```
close(fd_temp);
```

```
off_t chunk_size = file_size / P;
```

Παράμετρος lseek	Τιμή	Σημασία
fd	fd_temp	Το αρχείο
offset	0	Μετακίνηση κατά 0 bytes
whence	SEEK_END	Από το τέλος του αρχείου

Επιστρέφει τη θέση μετά την κίνηση, δηλαδή το συνολικό μέγεθος του αρχείου σε bytes. Στη συνέχεια διαιρείται σε P ίσα chunks.

Δημιουργία Children & Παράλληλη Επεξεργασία

```
for (int i = 0; i < P; i++) {  
    pid_t pid = fork();  
    if (pid == 0) {  
        signal(SIGINT, SIG_IGN); // Child αγνοεί Ctrl+C  
        close(pipefd[0]);        // Child δεν χρειάζεται read end  
        sleep(5);                // Παράθυρο για Ctrl+C demo  
        ...  
    } else {  
        active_children++;  
    }  
}
```

Κατανομή Τμημάτων

```
off_t start = i * chunk_size;  
off_t end  = (i == P - 1) ? file_size : start + chunk_size;  
lseek(fd_in, start, SEEK_SET);
```

Ανάγνωση Chunk

```
off_t bytes_to_read = end - start;  
off_t total_read = 0;  
  
while (total_read < bytes_to_read) {  
    size_t to_read = sizeof(buffer);  
    if (bytes_to_read - total_read < sizeof(buffer))  
        to_read = bytes_to_read - total_read;  
  
    bytes = read(fd_in, buffer, to_read);  
    ...  
}
```

```
total_read += bytes;
}
```

Parent - Συλλογή Αποτελεσμάτων

```
close(pipefd[1]); // Ο parent κλείνει το write end — ΚΡΙΣΙΜΟ
int total_count = 0;
int partial_count;
for (int i = 0; i < P; i++) {
    if (read(pipefd[0], &partial_count, sizeof(int)) > 0)
        total_count += partial_count;
}
close(pipefd[0]);
```

Ο parent πρέπει να κλείσει το pipefd[1]. Αν δεν το κάνει, το pipe δεν θα φτάσει ποτέ σε EOF και η read() θα μπλοκάρει για πάντα. Στη συνέχεια ο parent αθροίζει τα P μερικά αποτελέσματα.

wait() & Καθαρισμός

```
while ((wpid = wait(&status)) > 0)
    active_children--;
```

Ο parent περιμένει **όλα** τα children να τερματίσουν (όχι μόνο ένα), αποτρέποντας zombie processes, και ενημερώνει τον active_children counter.

- System Calls που χρησιμοποιεί ο κώδικας
 - pipe()
 - lseek()
 - sigaction()
 - signal()
 - sleep()

Μέρος 4

- Ζητούμενο:

Το ζητούμενο είναι η ανάπτυξη μιας παράλληλης εφαρμογής καταμέτρησης χαρακτήρων σε αρχείο, η οποία οργανώνεται σε τρία ανεξάρτητα επίπεδα διεργασιών: το Front-end για τη λήψη εντολών από τον χρήστη, τον Dispatcher για τον συντονισμό και την κυκλική κατανομή του φόρτου, και τους δυναμικά μεταβαλλόμενους Workers που αναλαμβάνουν το πρακτικό κομμάτι της μέτρησης. Η επικοινωνία μεταξύ των διεργασιών πρέπει να γίνεται αποκλειστικά μέσω Pipes (με χρήση σημάτων για την αναφορά προόδου), ενώ η εφαρμογή απαιτείται να είναι ανθεκτική σε σφάλματα, διασφαλίζοντας ότι αν ένας Worker τερματιστεί απροσδόκητα, ο Dispatcher θα του πάρει την ημιτελή εργασία και θα την αναθέσει σε άλλον εργάτη ώστε να ολοκληρωθεί ομαλά η αναζήτηση.

- Υλοποίηση Μέρους 3

Η υλοποίηση του μέρους 3 της άσκησης περιλαμβάνει δύο ξεχωριστά αρχεία κώδικα C. Το αρχείο `main.c` και το αρχείο `worker.c`

- Αρχείο: `main.c`

Ο κώδικας του αρχείου `main.c` αποτελεί την πλήρη υλοποίηση σε γλώσσα C του συστήματος Front-end και Dispatcher για την παράλληλη εφαρμογή καταμέτρησης χαρακτήρων που περιγράφηκε στο προηγούμενο κείμενο.

Ουσιαστικά, συντονίζει την επικοινωνία με τον χρήστη και τη διαχείριση των εργατών (workers).

1. Η συνάρτηση `main` (Αρχικοποίηση)

- Ξεκινάει ζητώντας το όνομα του αρχείου ως όρισμα από τη γραμμή εντολών.
- Δημιουργεί δύο ανώνυμα Pipes για την αμφίδρομη επικοινωνία μεταξύ Front-end και Dispatcher.
- Κάνει `fork()`: η θυγατρική διεργασία τρέχει τη συνάρτηση `run_dispatcher` και η γονική τη `run_frontend`.

2. Το Front-End (`run_frontend`)

Αυτή η διεργασία αλληλεπιδρά με τον χρήστη.

- Χρησιμοποιεί την κλήση συστήματος `select()`, η οποία της επιτρέπει να περιμένει ταυτόχρονα δεδομένα από δύο πηγές: το πληκτρολόγιο (εντολές χρήστη) και το `pipe` του Dispatcher (απαντήσεις/αποτελέσματα), χωρίς να "κολλάει" (block) σε κανένα από τα δύο.
- Διαχείριση εντολών:
 - `add, search <char>, exit`: Τις μετατρέπει σε κατάλληλα μηνύματα (π.χ. "ADD", "SEARCH c") και τις στέλνει μέσω `pipe` στον Dispatcher.
 - `status`: Δεν στέλνει μήνυμα μέσω `pipe`. Αντίθετα, στέλνει το σήμα `SIGUSR1` στον Dispatcher, ικανοποιώντας την απαίτηση της άσκησης για διακοπή μέσω σήματος.

3. Ο Dispatcher (`run_dispatcher`)

Αυτή είναι η "καρδιά" του συστήματος. Εκτελείται σε έναν ατέρμονα βρόχο (`while(1)`) ελέγχοντας συνεχώς, με μη-μπλοκάροντα (non-blocking) τρόπο, διάφορα γεγονότα:

- Ανθεκτικότητα (Fault Tolerance): Χρησιμοποιεί την `waitpid(-1, ..., WNOHANG)` για να δει αν κάποιος `worker` "πέθανε" (π.χ. τερματίστηκε βίαια). Αν εντοπίσει νεκρό `worker`, μαρκάρει το κομμάτι του αρχείου (`chunk`) που του είχε αναθέσει ξανά ως "μη ανατεθειμένο" (`status = 0`), ώστε να δοθεί σε άλλον εργάτη.
- Αναφορά Προόδου: Αν δεχτεί το σήμα `SIGUSR1`, ενεργοποιείται μια μεταβλητή (`flag`). Ο Dispatcher το βλέπει, υπολογίζει το ποσοστό (% ολοκλήρωσης = `completed chunks / total chunks`) και στέλνει την αναφορά στο Front-end.
- Διαχείριση Εντολών:
 - Αν λάβει `ADD`: Κάνει `fork()` και στη συνέχεια `execv("./worker", ...)` για να μετατρέψει τη νέα διεργασία στο εκτελέσιμο του εργάτη. Της περνάει ως ορίσματα τα `file descriptors` των `pipes` για να ξέρει πού να γράφει.
 - Αν λάβει `SEARCH`: Διαβάζει το μέγεθος του αρχείου με τη `stat()`, το "κόβει" σε λογικά `chunks` των 1024 bytes και αρχικοποιεί τον πίνακα `chunks`.
- Κατανομή Εργασίας: Διατρέχει τον πίνακα με τα `chunks`. Αν βρει ελεύθερο `chunk` και ελεύθερο `worker`, του στέλνει μήνυμα `TSK <offset> <size> <char>`.
- Συλλογή Αποτελεσμάτων: Διαβάζει απαντήσεις τύπου `RES <worker_id> <βρέθηκαν>` από τους `workers`, αθροίζει τους χαρακτήρες, ελευθερώνει τον `worker` και μαρκάρει το `chunk` ως ολοκληρωμένο.

- Αρχείο: worker.c

Ο συγκεκριμένος κώδικας προορίζεται να μεταγλωττιστεί σε ένα αυτόνομο εκτελέσιμο (π.χ., με `gcc worker.c -o worker`), το οποίο καλείται και ελέγχεται δυναμικά από τον Dispatcher μέσω της εντολής `execv()`.

Η Λειτουργία του Worker

1. Αρχικοποίηση μέσω Ορισμάτων:

Όταν ο Dispatcher "γεννάει" (fork) αυτόν τον worker, του περνάει 4 ορίσματα:

- ο Το ID του εργάτη.
- ο Το File Descriptor Εισόδου (`fd_in`): Το άκρο του pipe από όπου θα διαβάζει εντολές.
- ο Το File Descriptor Εξόδου (`fd_out`): Το άκρο του pipe όπου θα στέλνει τα αποτελέσματα.
- ο Το Όνομα του αρχείου που θα αναλυθεί (το οποίο ανοίγει αμέσως για ανάγνωση).

2. Ατέρμονος Βρόχος Εργασίας (Event Loop):

Ο Worker δεν τερματίζει από μόνος του. Μπαίνει σε μια λούπα `while(read(...))` περιμένοντας να λάβει δεδομένα από τον Dispatcher. Αν το pipe κλείσει (π.χ. αν ο Dispatcher στείλει SIGKILL με την εντολή `exit`), η `read` αποτυγχάνει και ο Worker τερματίζει.

3. Εκτέλεση του Chunk (Αναζήτηση):

Μόλις λάβει ένα μήνυμα εργασίας (μορφής TSK <offset> <size> <char>):

- ο Χρησιμοποιεί την κλήση `lseek(file_fd, offset, SEEK_SET)` για να μετακινήσει τον "κέρσορα" του αρχείου ακριβώς στο σημείο (σε bytes) που του ζητήθηκε.
- ο Ξεκινάει να διαβάζει το κομμάτι του αρχείου σε δικά του, μικρότερα τοπικά chunks των 1024 bytes (για αποδοτικότητα μνήμης), μέχρι να συμπληρώσει το συνολικό size που του ανατέθηκε.
- ο Καταμετρά τις εμφανίσεις του χαρακτήρα-στόχου.

4. Προσομοίωση Φόρτου (usleep):

Η εντολή `usleep(500000)` "κοιμίζει" τον Worker για μισό δευτερόλεπτο (500 ms) μετά από κάθε chunk. Αυτή είναι μια πολύ έξυπνη προσθήκη: επειδή οι σύγχρονοι υπολογιστές είναι πολύ γρήγοροι, η αναζήτηση θα τελείωνε ακαριαία. Η καθυστέρηση αυτή βάζει ένα "τεχνητό φρένο" ώστε να προλαβαίνεις να χρησιμοποιήσεις την εντολή `status` στο Front-end και να δεις την παράλληλη εκτέλεση στην πράξη!

5. Επιστροφή Αποτελέσματος:

Μόλις τελειώσει τον υπολογισμό, στέλνει ένα τυποποιημένο μήνυμα των 16 χαρακτήρων (π.χ. `RES 00002 00045`, δηλαδή "Ο Worker 2 βρήκε 45 εμφανίσεις") πίσω στον Dispatcher μέσω του `fd_out`.

Σε συνδυασμό με το προηγούμενο αρχείο, η εφαρμογή σου ικανοποιεί πλήρως όλες τις απαιτήσεις του κειμένου.

- Πετυχαίνει τον στόχο του;

ΝΑΙ!